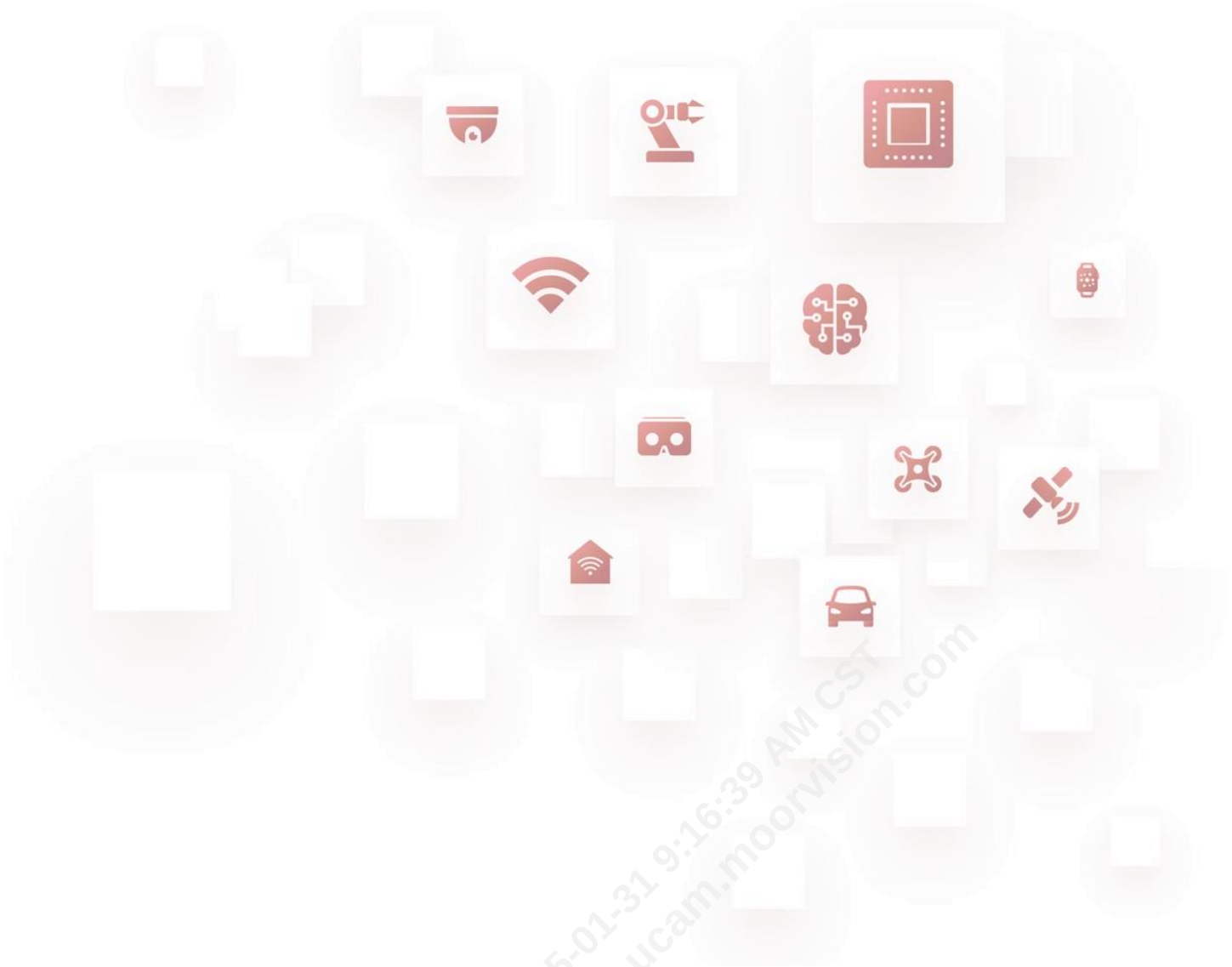
Thundercomm
AIOT ENABLER



2025-01-31 9:16:39 AM CST
Steven_ucam.moorvision.com

Thundercomm TurboX C410/C610 Linux Embedded

SNPE2.X User Manual

Rev. V1.0
Dec 21, 2023

DN: tc-g-23210

Revision History

Revision	Date	Author	Description
1.0	Dec 21, 2023	Lei Meng	Initial release.

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

Table List

Table 2-1. Hexagon SDK architecture and versions

Table 3-1. Supported devices

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

About this document

- Illustrations in this documentation might look different from your product.
- Depending on the model, some optional accessories, features, and software programs might not be available on your device.
- Depending on the version of operating systems and programs, some user interface instructions might not be applicable to your device.
- Documentation content is subject to change without notice. Thundercomm makes constant improvements on the documentation of the products, including this guidebook.
- Function declarations, function names, type declarations, attributes, and code samples appear in a different format, for example, `cp armcc armcpp`.
- Code variables appear in angle brackets, for example, `<number>`.
- Button, tool, and key names appear in bold font, for example, click **Save** or press **Enter**.
- Commands to be entered appear in a different font; on the host computer use $ as shell prompt, while on the target device use # as shell prompt, for example,

```
$ adb devices
# logcat
```

- Part of the code that does not contain instructions appear in a different format, for example,

```
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0777", GROUP="adm"
```

- Folders, path and files are formatted in italic, for example, *turbox_flash_flat.sh*.

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

Table of Contents

Chapter 1. Overview ..... - 2 -
1.1. Capabilities ..... - 2 -
1.2. Model workflow ..... - 3 -
Chapter 2. Setup ..... - 4 -
2.1. Download SNPE ..... - 4 -
2.2. SNPE setup ..... - 4 -
Chapter 3. Network Models ..... - 7 -
3.1. Supported devices ..... - 7 -
3.2. Model conversion ..... - 7 -
3.3. Quantize the model ..... - 8 -
Chapter 4. Device Validation ..... - 9 -
Chapter 5. Running Model ..... - 10 -
5.1. On the host ..... - 10 -
5.2. On the device ..... - 11 -
Chapter 6. Benchmark ..... - 13 -
6.1. Prerequisites ..... - 13 -
6.2. Run the benchmark ..... - 14 -
Chapter 7. Code Example ..... - 15 -
7.1. Prerequisites ..... - 15 -
7.2. Build the C++ application ..... - 15 -
Appendix 1. Notices ..... - 16 -
Appendix 2. Trademarks ..... - 18 -

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
stefan_ucam.modvision.com

Chapter 1. Overview

1.1. Capabilities

The SNPE (Qualcomm® Neural Processing SDK) is a Qualcomm Snapdragon software accelerated runtime for the execution of deep neural networks. With SNPE, users can:

- Execute an arbitrarily deep neural network.
- Execute the network on the Snapdragon CPU, the Adreno GPU or the Hexagon DSP.
- Debug the network execution on x86 Ubuntu Linux.
- Convert PyTorch, TFLite, ONNX, and TensorFlow models to a Qualcomm® Neural Processing SDK DLC (Deep Learning Container) file.
- Quantize DLC files to 8-bit or 16-bit fixed point for running on the Hexagon DSP.
- Debug and analyze the performance of the network with Qualcomm® Neural Processing SDK tools.

This document provides detailed instructions on downloading, setting up, and operating SNPE2.X on the TurboX C410/C610 platform running Linux Embedded.

➲ "TurboX" referred to herein is the English text of our registered trademark: TURBOX.

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

1.2. Model workflow

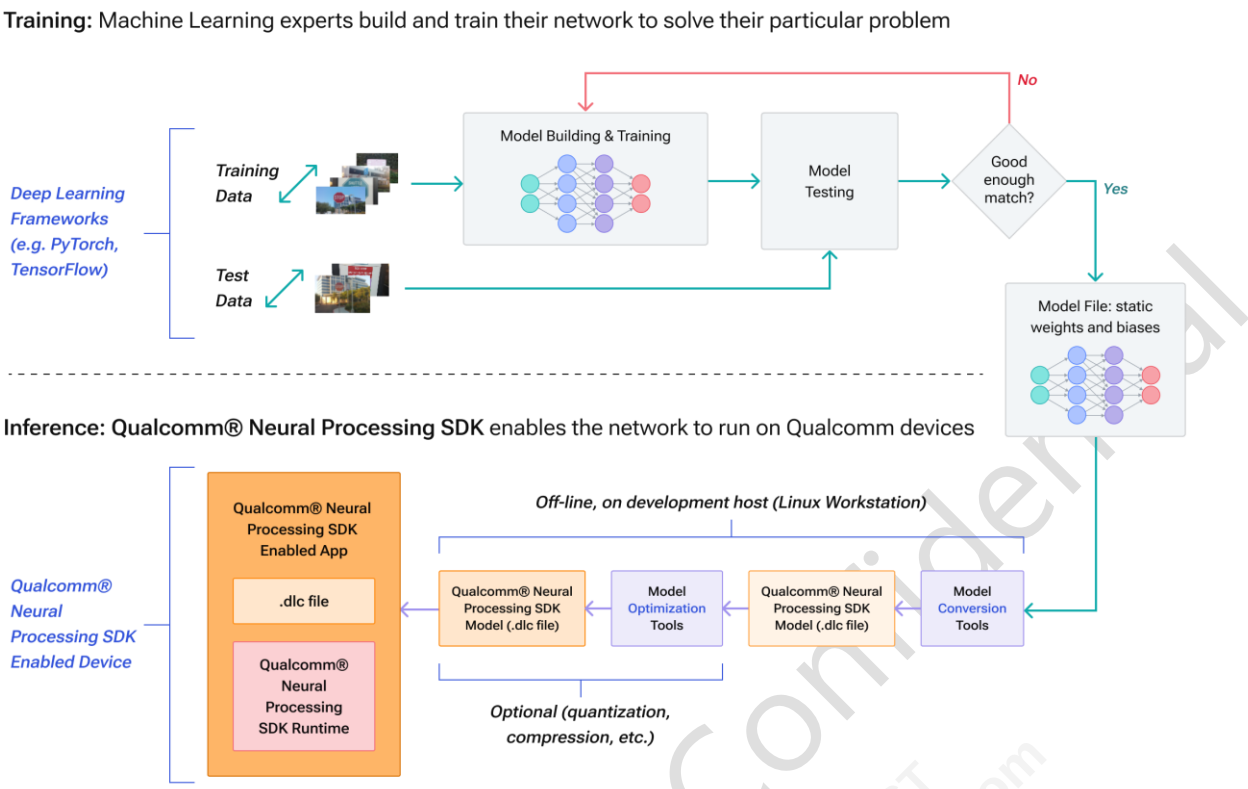


Figure 1-1.Model Workflow

Model training is performed on a popular deep learning framework (PyTorch, TFLite, ONNX, and TensorFlow models are supported by SNPE.) After training is completed, the trained model is converted into a DLC file that can be loaded into the SNPE runtime. This DLC file can then be used to perform forward inference passes using one of the Snapdragon accelerated compute cores.

The basic SNPE workflow only consists of several steps:

Step 1.Convert the network model to a DLC file that can be loaded by SNPE.

Step 2.Optionally quantize the DLC file for running on the Hexagon DSP.

Step 3.Prepare input data for the model.

Step 4.Load and execute the model using SNPE runtime.

Thundercomm Confidential

Chapter 2. Setup

2.1. Download SNPE

Download the SNPE installation package and unzip it to the desired location (for example, "*/opt/qcom/aistack/snpe/2.13.0.230730*" in this document).

2.2. SNPE setup

2.2.1. Host OS requirement

The SNPE SDK development environment is currently limited to Ubuntu, specifically version 18.04 and 20.04.

2.2.2. Set up Python environment

The SNPE SDK distribution is supported on Python 3 only. Download and installation instructions vary based on the host OS. The SDK has been tested with Python 3.8.

Step 1. Download the latest version of Anaconda.

Step 2. Execute the following commands.

```
$ chmod +x Anaconda3-<version>-Linux-x86_64.sh
$ bash Anaconda3-<version>-Linux-x86_64.sh
```

Step 3. Enter the installation path of Anaconda (default path: "*~/anaconda3*") and add the environment variables for anaconda in *~/.bashrc.*

```
$ export PATH=~/anaconda3/bin:$PATH
$ source ~/.bashrc
```

Step 4. Verify whether the installation is successful.

```
$ conda -h 2>/dev/null && echo "Success" || echo "Failure"
```

Step 5. Create a Python virtual environment.

```
$ conda create -n snpe python=3.8
```

Step 6. Enter the virtual environment.

```
$ conda activate snpe
```

Afterwards, all SNPE work will be done in the virtual environment.

➲ **NOTE**: If you need to exit the virtual environment, execute the following command:

```
$ conda deactivate
```

Thundercomm Confidential
2025-01-31 16:39 AM CST
Stefan_ucam.moorvision.com

2.2.3. Set environment variables

Step 1. Navigate to the SNPE SDK root directory (default: */opt/qcom/aistack/snpe/2.xx*).

Step 2. Set the user environment with the provided *envsetup.sh* script.

```
$ source ${SNPE_ROOT}/bin/envsetup.sh
```

Then the following environment variables will be set/updated:

- PYTHONPATH
- PATH
- LD_LIBRARY_PATH
- SNPE_ROOT: represents the full path to the SNPE SDK root.

  ➲ **NOTES:**

  - SNPE API headers are stored in *${SNPE_ROOT}/include/SNPE*.
  - The SNPE tools are stored in *${SNPE_ROOT}/bin/x86_64-linux-clang*.
  - Target specific libraries are stored in *${SNPE_ROOT}/lib/*/*.

2.2.4. Check additional packages

Check and install missing dependencies with the following commands:

```
$ python3 -m pip install --upgrade pip
$ chmod 777 ${SNPE_ROOT}/bin/check-python-dependency
$ ${SNPE_ROOT}/bin/check-python-dependency
```

2.2.5. make

Operation packages are compiled with a frontend that is written with Makefiles. If **make** is unavailable on your host machine, install it with the following commands:

```
$ sudo apt-get update
$ sudo apt-get install make
```

2.2.6. Clang-9

Compiling artifacts for the x86_64 target requires **clang++**. This SNPE release is verified to work with **clang-9**.

Run the following script to check and install the missing Linux dependencies:

```
$ sudo bash ${SNPE_ROOT}/bin/check-linux-dependency.sh
```

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
shifan.cam.moonvision.com

2.2.7. Hexagon SDK installation on Linux

Compiling for DSP requires the use of the Hexagon toolchain available from Qualcomm® Hexagon SDK.

Table 2-1. Hexagon SDK architecture and versions

Backend	Hexagon Architecture	Hexagon SDK Version *	Hexagon Tools Version
DSP	V73	5.0.0	8.6.02
DSP	V69	4.3.0	8.5.03
DSP	V68	4.2.0	8.4.09
DSP	V65	3.5.2	8.3.07
DSP	V66	4.1.0	8.4.06

* You can download the desired Hexagon SDK version via the corresponding link in this column. For detailed usage, please unzip the downloaded file and refer to the tutorial under the *${Hexagon_ROOT}/docs* path.

2.2.8. Set up ML framework

- **TensorFlow:**

  Step 1. Please install TensorFlow (CPU version) as a standalone Python module from:
  https://pypi.org/project/tensorflow/2.10.1/

  Step 2. Check the version of TensorFlow installed.

```
$ python3 -c "import tensorflow as tf; print('Tensorflow Version:',
tf.__version__)"
  Tensorflow Version: 2.10.1
```

  Step 3. Add required environment variables.

```
$ # Do some setting
$ pip show tensorflow | grep Location
  Location: /home/username/anaconda3/envs/snpe/lib/python3.8/site-packages
$ export TENSORFLOW_HOME=/home/username/anaconda3/envs/snpe/lib/python3.8/site-
packages/tensorflow
$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/username/anaconda3/envs/snpe/lib
```

- **ONNX:**

  Step 1. Install ONNX as a standalone Python module.

  Step 2. Check whether TFLite is installed successfully.

```
$ python3 -c "import onnx; print('ONNX Version:', onnx.__version__)"
  ONNX Version: 1.11.0
```

- **TFLite:**

  Step 1. Install TFLite as a standalone Python module.

  Step 2. Check whether TFLite is installed successfully.

```
$ python3 -c "import tflite"
```

- **PyTorch:**

  Step 1. Install PyTorch v1.8.1 as a standalone Python module.

  Step 2. Check whether PyTorch is installed successfully.

```
$ python3 -c "import torch"
```

Thundercomm Confidential

Chapter 3. Network Models

3.1. Supported devices

Table 3-1. Supported devices

SOM device	Snapdragon Device/Chip	Supported Tool Chains	CPU	GPU	DSP	AIP
C610	QCS610LE	aarch64-oe-linux-gcc9.3	Yes	Yes	V66	No
C410	QCS410LE	aarch64-oe-linux-gcc9.3	Yes	Yes	V66	No

3.2. Model conversion

Machine Learning frameworks have specific formats for storing neural network models. SNPE supports these various models by converting them to a framework DLC format. The DLC file is used by the SNPE runtime for execution of the neural network.

This section takes TensorFlow model as an example to show the model transformation process.

For model transformation of other frameworks, refer to:

https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-2/usergroup1.html

Step 1. Download the TensorFlow model.

```
cd ${SNPE_ROOT}/examples/Models/inception_v3
sudo apt-get update
sudo apt-get install wget
wget -N --directory-prefix=./tensorflow
https://storage.googleapis.com/download.tensorflow.org/models/inception_v3_2016_08_2
8_frozen.pb.tar.gz
```

Step 2. Convert the model manually.

The **snpe-tensorflow-to-dlc** tool converts a frozen TensorFlow model or a graph meta file into an equivalent SNPE DLC file. The following command will convert an Inception v3 TensorFlow model into a SNPE DLC file.

```
$ snpe-tensorflow-to-dlc \
--input_network tensorflow/inception_v3_2016_08_28_frozen.pb \
--input_dim input "1,299,299,3" \
--out_node "InceptionV3/Predictions/Reshape_1" \
--output_path $SNPE_ROOT/examples/Models/inception_v3/dlc/inception_v3.dlc
```

Step 3. Obtain input data for quantization.

1) Convert the format of the input data from *.jpeg* to *.raw*.

```
$ cd $SNPE_ROOT/examples/Models/inception_v3/
$ mkdir -p data/cropped
$ python scripts/create_inceptionv3_raws.py -i data/ -d data/cropped/ -s 299
```

2) Create the list file.

```
$ python scripts/create_file_list.py -i data/cropped/ -o data/cropped/raw_list.txt
-e *.raw
$ python scripts/create_file_list.py -i data/cropped/ -o data/target_raw_list.txt
-e *.raw -r
```

Thundercomm Confidential

➲ **NOTE:** Alternatively, you can run the following commands to realize automatic conversion of the TensorFlow model:

```
$ cd $SNPE_ROOT/examples/Models/inception_v3
$ sudo apt-get install wget
$ python scripts/setup_inceptionv3.py -a . -d
```

Then the TensorFlow model will be converted automatically:

- The *inception_v3_2016_08_28_frozen.pb* is downloaded to:

  $SNPE_ROOT/examples/Models/inception_v3/tensorflow

- The converted DLC model file is located in *$SNPE_ROOT/examples/Models/inception_v3/dlc*.
- The input data for quantization is located in *$SNPE_ROOT/examples/Models/inception_v3/data/cropped.*

3.3. Quantize the model

Sample commands for converting an *Inception v3 DLC* file into a quantized *Inception v3 DLC* file:

```
$ cd $SNPE_ROOT/examples/Models/inception_v3/
$ snpe-dlc-quantize  --input_dlc dlc/inception_v3.dlc \
--input_list data/cropped/raw_list.txt \
--output_dlc dlc/inception_v3_quantized.dlc
```

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Chapter 4. Device Validation

The snpe-platform-validator tool is used to assess the SNPE compatibility and capability of a device. This tool is designed to run directly on the device itself, as opposed to on the host. In addition to its own executable file, the tool necessitates a few additional files to be transferred to the device.

Step 1. Push SNPE tools and libs.

```
$ export SNPE_TARGET_ARCH=aarch64-oe-linux-gcc9.3
$ adb shell "mkdir -p /data/local/tmp/platformValidator/bin"
$ adb shell "mkdir -p /data/local/tmp/platformValidator/lib"
$ adb shell "mkdir -p /data/local/tmp/platformValidator/dsp"
$ adb push $SNPE_ROOT/lib/$SNPE_TARGET_ARCH/* /data/local/tmp/platformValidator/lib
$ adb push $SNPE_ROOT/lib/hexagon-v68/unsigned/* \
/data/local/tmp/platformValidator/dsp
$ adb push $SNPE_ROOT/bin/$SNPE_TARGET_ARCH/snpe-platform-validator \
/data/local/tmp/platformValidator/bin
```

Step 2. Validate the device.

```
$ adb shell
# export LD_LIBRARY_PATH=/data/local/tmp/platformValidator/lib/
# export
ADSP_LIBRARY_PATH='/system/lib/rfsa/adsp;/system/vendor/lib/rfsa/adsp;/usr/lib/rfsa/
adsp;/data/local/tmp/platformValidator/dsp/'
# /data/local/tmp/platformValidator/bin/snpe-platform-validator --runtime all
```

Refer to the following validation result:

```
WARNING: SNPE AIP runtime not available on this SoC
PF_VALIDATOR: DEBUG: Calling PlatformValidator->setRuntime
PF_VALIDATOR: DEBUG: Calling PlatformValidator->RuntimeCheck
PF_VALIDATOR: DEBUG: Building and running a simple Vector addition gpu program.
Unit Test on the runtime GPU: Passed.
SNPE is supported for runtime GPU on the device.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->IsRuntimeAvailable
Runtime GPU Prerequisites: Present.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->setRuntime
PF_VALIDATOR: DEBUG: Calling PlatformValidator->RuntimeCheck
PF_VALIDATOR: DEBUG: Setting up QnnBackend
PF_VALIDATOR: DEBUG: Snpe-QNN HTP backend initialization successful
PF_VALIDATOR: DEBUG: CPU side validation passed.

PF_VALIDATOR: DEBUG: starting calculator test
PF_VALIDATOR: DEBUG: Loading HTP stub: libcalculator_htp.so
PF_VALIDATOR: DEBUG: Successfully loaded DSP library - 'libcalculator_htp.so'.
Setting up pointers.
PF_VALIDATOR: DEBUG: Success in executing the sum function
Unit Test on the runtime DSP: Passed.
SNPE is supported for runtime DSP on the device.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->IsRuntimeAvailable
PF_VALIDATOR: DEBUG: CPU side validation passed.

PF_VALIDATOR: DEBUG: starting calculator test
PF_VALIDATOR: DEBUG: Loading HTP stub: libcalculator_htp.so
PF_VALIDATOR: DEBUG: Successfully loaded DSP library - 'libcalculator_htp.so'.
Setting up pointers.
PF_VALIDATOR: DEBUG: Success in executing the sum function
Runtime DSP Prerequisites: Present.
Skipping checks for AIP runtime.
```

➲ **NOTE:** Validating the device with **snpe-platform-validator** and **snpe-platform-validator-py** will generate the same result. The above result "`Present`" indicates that GPU and DSP are present (OK); if the result appears "`Absent`", it indicates that the device is NG.

Chapter 5. Running Model

The example C++ application in this document is referred to as "**snpe-net-run**". It is a command line that executes a neural network with SNPE SDK APIs. Refer to the following parameters for **snpe-net-run**:

- **Required parameters:**
  - A neural network model (DLC file).
  - An input list file with paths to the input data.
- **Optional parameters:**
  - Choice of GPU, DSP runtimes (default: CPU).
  - Output directory (default: *./output*).
  - Show help description.

5.1. On the host

Step 1. Verify whether the converted model is functioning properly.

```
$ cd $SNPE_ROOT/examples/Models/inception_v3/
$ snpe-net-run --container dlc/inception_v3_quantized.dlc --input_list
data/cropped/raw_list.txt
```

Upon successful execution, the following output will be displayed and the *output/* directory will be generated in the current path.

```
--------------------------------------------------------------------------------
Model String: N/A
SNPE v2.13.0.230725130242_60412
--------------------------------------------------------------------------------
Processing DNN input(s):
/opt/qcom/aistack/snpe/2.13.0.230730/examples/Models/inception_v3/data/cropped/notic
e_sign.raw
Processing DNN input(s):
/opt/qcom/aistack/snpe/2.13.0.230730/examples/Models/inception_v3/data/cropped/trash
_bin.raw
Processing DNN input(s):
/opt/qcom/aistack/snpe/2.13.0.230730/examples/Models/inception_v3/data/cropped/plast
ic_cup.raw
Processing DNN input(s):
/opt/qcom/aistack/snpe/2.13.0.230730/examples/Models/inception_v3/data/cropped/chair
s.raw
Successfully executed!
```

Step 2. View the classification result.

```
$ cd $SNPE_ROOT/examples/Models/inception_v3
$ python scripts/show_inceptionv3_classifications.py \
-i data/target_raw_list.txt -o output \
-l data/imagenet_slim_labels.txt
```

Refer to the following classification result:

```
Classification results
cropped/notice_sign.raw 0.170401 459 brass
cropped/trash_bin.raw   0.747274 413 ashcan
cropped/plastic_cup.raw 0.977711 648 measuring cup
cropped/chairs.raw     0.299139 832 studio couch
```

Thundercomm Confidential 2025-01-31 10:16:19 AM CST Stefan_ucam.moonvision.com

5.2. On the device

Step 1. Push SNPE tools and libs.

```
$ export SNPE_TARGET_ARCH=aarch64-oe-linux-gcc9.3
$ adb shell "mkdir -p /data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/bin"
$ adb shell "mkdir -p /data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/lib"
$ adb shell "mkdir -p /data/local/tmp/snpeexample/dsp/lib"
$ adb push $SNPE_ROOT/lib/$SNPE_TARGET_ARCH/* \
/data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/lib
$ adb push $SNPE_ROOT/lib/hexagon-v66/unsigned/* /data/local/tmp/snpeexample/dsp/lib
$ adb push $SNPE_ROOT/bin/$SNPE_TARGET_ARCH/snpe-net-run \
/data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/bin
```

Step 2. Push *inception_v3*.

```
$ cd $SNPE_ROOT/examples/Models/inception_v3/
$ adb shell "mkdir -p /data/local/tmp/inception_v3/cropped"
$ adb push data/cropped/*raw /data/local/tmp/inception_v3/cropped
$ adb push data/target_raw_list.txt /data/local/tmp/inception_v3
$ adb push dlc/inception_v3.dlc /data/local/tmp/inception_v3
```

Step 3. Run the following commands on the device.

```
$ adb shell
# export LD_LIBRARY_PATH=/data/local/tmp/snpeexample/aarch64-oe-linux-
gcc9.3/lib:$LD_LIBRARY_PATH
# export ADSP_LIBRARY_PATH="/data/local/tmp/snpeexample/aarch64-oe-linux-
gcc9.3/lib/../../dsp\
/lib;/system/lib/rfsa/adsp;/usr/lib/rfsa/adsp;/system/vendor/lib/rfsa/adsp;/dsp;/etc
/images/dsp;"
# cd /data/local/tmp/inception_v3
# rm -rf output
# /data/local/tmp/snpeexample/aarch64-oe-linux-gcc9.3/bin/snpe-net-run --container
inception_v3.dlc \
 --input_list target_raw_list.txt
```

➲ **NOTE**: **snpe-net-run** has parameters --use_gpu' and '--use_dsp' for GPU and DSP respectively. Refer to the following example:

```
# snpe-net-run --container inception_v3.dlc --input_list target_raw_list.txt
# snpe-net-run --container inception_v3.dlc --input_list target_raw_list.txt --
use_gpu
# snpe-net-run --container inception_v3_quantized.dlc --input_list
target_raw_list.txt --use_dsp
```

Refer to the following log output:

```
--------------------------------------------------------------------------------
Model String: N/A
SNPE v2.13.0.230725130242_60412
--------------------------------------------------------------------------------
Processing DNN input(s):
cropped/notice_sign.raw
Processing DNN input(s):
cropped/trash_bin.raw
Processing DNN input(s):
cropped/plastic_cup.raw
Processing DNN input(s):
cropped/chairs.raw
Successfully executed!
```

Step 4. Pull the generated result (the generated output directory) to the host.

Step 5. Execute the classification commands.

```
$ cd $SNPE_ROOT/examples/Models/inception_v3
$ mkdir output
$ adb pull /data/local/tmp/inception_v3/output .
$ python scripts/show_inceptionv3_classifications.py \
-i data/target_raw_list.txt -o output \
-l data/imagenet_slim_labels.txt
```

The following classification result will be generated:

```
Classification results
cropped/notice_sign.raw 0.170401 459 brass
cropped/trash_bin.raw   0.747274 413 ashcan
cropped/plastic_cup.raw 0.977711 648 measuring cup
cropped/chairs.raw     0.299139 832 studio couch
```

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Chapter 6. Benchmark

The benchmark shipped in the SNPE SDK consists of a set of Python scripts that run a network on a target Android/Linux Embedded device and collect performance metrics. It uses executables and libraries found in the SDK package to run a DLC file on target, as well as a set of inputs for the network and a file that points to that set of inputs.

The input of the benchmark scripts is a configuration file in JSON format. The SDK ships with a configuration file for running the InceptionV3 model that is created in the SNPE SDK. SDK users are encouraged to create their own configuration files and use the benchmark scripts to run on the target to collect timing and memory consumption measurements.

The configuration file allows the user to specify:

- Name of the benchmark (i.e., ***InceptionV3***)
- Host path for storing results
- Device paths to use (where to push the necessary files for running the benchmark)
- Device to run the benchmark on (only one device is supported per run)
- Hostname/IP of remote machine to which devices are connected
- Number of times to repeat the run
- Model specific information (name, location of DLC, location of inputs)
- SNPE runtime configuration(s) to use (combination of CPU, GPU, GPU_FP16 and DSP)
- Measurements to take ("mem" and/or "timing")
- Profiling level of measurements ("off", "basic", "moderate" or "detailed")

6.1. Prerequisites

- SNPE setup (source bin/envsetup...)
- *model.dlc*
- Input data (list)

➲ **NOTE:** When the following error log occurs on DSP, it indicates the absence of a signature.

```
INFO - snpe_bench: Run 1
2022-12-07 13:45:02,851 - ERROR - snpebm.snpebm_device: ['SNPEFactory error 908: Error
code: -2. rpc_error_code=0x72, line_no=334', 'The selected runtime is not available on
this platform. Continue anyway to observe the failure at network creation time.']
2022-12-07 13:45:02,852 - ERROR - snpebm.snpebm_device: ['error_code=908;
error_message=DSP runtime communication error. Error code: -2. rpc_error_code=0x72,
line_no=334; error_component=DSP Runtime; line_no=417; thread_id=-159218576',
'error_code=908; error_message=DSP runtime communication error. Error code: -2.
rpc_error_code=0x72, line_no=334; error_component=DSP Runtime; line_no=417; thread_id=-
159218576']
2022-12-07 13:45:02,852 - WARNING - snpe_bench: Failed to perform benchmark for
DSP_timing.
2022-12-07 13:45:02,852 - INFO - snpe_bench: Running on DSP_ub_float_timing
```

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
ucam.moorvision.com

6.2. Run the benchmark

Step 1. Create the configuration file.

```
$ cd $SNPE_ROOT/benchmarks/SNPE
$ vi inception_v3.json
```

Step 2. Copy the following contents into the *inception_v3.json* file.

```
{
    "Name":"Inception_v3",
    "HostRootPath": "inception_v3",
    "HostResultsDir":"inception_v3/results",
    "DevicePath":"/data/local/tmp/snpebm",
    "Devices":["6bc6f90c"],
    "HostName": "localhost",
    "Runs":2,

    "Model": {
        "Name": "INCEPTION_V3",
        "Dlc": "../../examples/Models/inception_v3/dlc/inception_v3_quantized.dlc",
        "InputList": "../../examples/Models/inception_v3/data/target_raw_list.txt",
        "Data": [
            "../../examples/Models/inception_v3/data/cropped"
            ]
    },

    "Runtimes":["DSP"],
    "Measurements": ["timing"]

 }
```

➲ **NOTE**: Some paths can be modified as required.

Step 3. Run the benchmark.

```
$ python snpe_bench.py -c inception_v3.json -t le64_oe_gcc9.3
```

Step 4. The result generated in the *$SNPE_ROOT/benchmark/inception_v3/results/latest_results* can be checked in **Excel** or **LibreOffice calc**.

0	avg (us)	max (us)	min (us)	runtime	avg (us)	max (us)	min (us)	runtime
Load	198	199	198	CPU	199	208	191	CPU
Deserialize	67899	68189	67610	CPU	68924	69617	68232	CPU
Create	75642	75694	75591	CPU	75795	75837	75754	CPU
Init	149949	150020	149879	CPU	151550	152248	150852	CPU
De-Init	12822	13260	12384	CPU	13616	14629	12603	CPU
Create Network(s)	71244	71255	71234	CPU	71426	71454	71399	CPU
RPC Init Time	0	0	0	CPU	0	0	0	CPU
Snpe Accelerator Init Time	0	0	0	CPU	0	0	0	CPU
Accelerator Init Time	0	0	0	CPU	0	0	0	CPU
Total Inference Time	342250	346366	338135	CPU	345712	346724	344701	CPU
Forward Propagate	342226	346342	338110	CPU	345685	346697	344674	CPU
RPC Execute	0	0	0	CPU	0	0	0	CPU
Snpe Accelerator	0	0	0	CPU	0	0	0	CPU
Accelerator	0	0	0	CPU	0	0	0	CPU
Misc Accelerator	0	0	0	CPU	0	0	0	CPU
layer_000 (Name:InceptionV3/InceptionV3/Conv2d_1a_3x3/convolution Type:Conv2d) (cycles)	3902	4020	3784	CPU	3540	3555	3526	CPU

Figure 6-1. Check Result in Excel or LibreOffice Calc

As shown in Figure 6-1, the CSV file contains the initialization time, reasoning time, and other associated metrics for the Inception_v3 model.

➲ **NOTE:** For more information on CSV files, refer to:

https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-2/benchmarking.html

Chapter 7. Code Example

This section demonstrates how to build a C++ sample application that can execute neural network models on the PC or target device.

7.1. Prerequisites

- Set up SNPE.
- Obtain *Inception_v3*.

7.2. Build the C++ application

Step 1. Move to the base directory of snpe-sample.

```
$ cd $SNPE_ROOT/examples/SNPE/NativeCpp/SampleCode
```

Step 2. Run the commands below to build snpe-sample.

```
$ sudo apt-get install gcc-aarch64-linux-gnu
$ sudo apt-get install g++-aarch64-linux-gnu
$ export CXX=aarch64-linux-gnu-g++
$ make -f Makefile.aarch64-oe-linux-gcc9.3
```

Step 3. The `make` command will build a binary file for a Linux Embedded snpe-sample.

$SNPE_ROOT/examples/NativeCpp/SampleCode/obj/local/aarch64-oe-linux-gcc9.3/snpe-sample

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Appendix 1. Notices

Thundercomm may have patents or pending patent programs covering subject matter described in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries to service@thundercomm.com.

THUNDERCOMM PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you.

Changes are made periodically to the information herein; these changes will be incorporated in new editions of the publication. To provide better service, Thundercomm reserves the right to improve and/or modify the products and software programs described in the manuals, and the content of the manual, at any time without additional notice.

The software interface and function and hardware configuration described in the manuals included with your development board or system on module might not match exactly the actual configuration of that you have purchased. For the configuration of the product, refer to the related contract (if any) or product packing list, or consult the distributor for the product sales. Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

The products described in this document are not intended for use in implantation or other life support applications where malfunction may result in injury or death to persons. The information contained in this document does not affect or change Thundercomm product specifications or warranties. Nothing in this document shall operate as an express or implied license or indemnity under the intellectual property rights of Thundercomm or third parties. All information contained in this document was obtained in specific environments and is presented as an illustration. The result obtained in other operating environments may vary.

The information of this document should not be as any invitation for offer or any advice to the visitors. Please consult the professional comments from the sales consultant prior to do any actions of invest or purchase.

Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Any references in this publication to non-Thundercomm Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this Thundercomm product, and use of those Web sites is at your own risk.Thundercomm shall not be responsible for the content of the third party.

Any performance data contained herein was determined in a controlled environment. Therefore, the result obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

This document is copyrighted by Thundercomm and the property right of the date mentioned in this document, including but not limited trademarks, patents, copyrights, trade name etc. is are not covered by any open-source license. Thundercomm may update this document at any time without notice.

Anyone doesn’t have the right to amend, reprint, republication, reproduce, transmit, distribute or any other way to use this document in business or public purpose without the prior written consent by Thundercomm.

E-mail messages sent to Thundercomm via the Internet are not guaranteed to be completely secure.Thundercomm shall not be liable for any loss incurred by the surfer when transmitting any information over the Internet or for any loss incurred by Thundercomm when sending any information over the Internet

Thundercomm Confidential

at your request.

Thundercomm has all rights under other relevant exemptions provided by laws and regulations, and Thundercomm's failure to claim or delay in claiming such rights shall not be deemed to be a waiver of such rights by Thundercomm.

Thundercomm reserves the right of final interpretation of this document.

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com

Appendix 2. Trademarks

Thundercomm, Thundercomm TurboX, TURBOX, Thundersoft turbox are trademarks of Thundercomm Corporation or its associate companies in China and/or other countries. Intel, Intel SpeedStep, Optane, and Thunderbolt are trademarks of Intel Corporation or its subsidiaries in the U.S. and/or other countries. Microsoft, Windows, Direct3D, BitLocker, and Cortana are trademarks of the Microsoft group of companies. Mini DisplayPort (mDP), DisplayPort, and VESA are trademarks of the Video Electronics Standards Association. The terms HDMI and HDMI High-Definition Multimedia Interface are trademarks or registered trademarks of HDMI Licensing LLC in the United States and other countries. Wi-Fi, Wi-Fi Alliance, WiGig, and Miracast are registered trademarks of Wi-Fi Alliance. USB-C is a registered trademark of USB Implementers Forum. All other trademarks are the property of their respective owners.

Thundercomm Confidential
2025-01-31 9:16:39 AM CST
Stefan_ucam.moorvision.com